Задание к лабораторной работе №1

«Классификация текста: от классического ML до дообучения трансформера и анализа переобучения»

Содержание

1 Описание задания **3**

1.1 Задание 1. Разведка данных и классический baseline 4

1.2 Задание 2. Дообучение предобученного трансформера 5

1.3 Задание 3. Анализ переобучения и ошибок 6

1.4 Содержание отчёта и оценивание 7

2 Данные для вариантов **7**

2.1 Вариант 1 10

2.2 Вариант 2 10

2.3 Вариант 3 11

2.4 Вариант 4 11

2.5 Вариант 5 12

2.6 Вариант 6 12

2.7 Вариант 7 13

2.8 Вариант 8 13

2.9 Вариант 9 14

2.10 Вариант 10 14

2.11 Вариант 11 15

2.12 Вариант 12 15

2.13 Вариант 13 16

2.14 Вариант 14 16

2.15 Вариант 15 17

2.16 Вариант 16 17

2.17 Вариант 17 18

2.18 Вариант 18 18

2.19 Вариант 19 19

2.20 Вариант 20 19

2.21 Вариант 21 20

2.22 Вариант 22 20

2.23 Вариант 23 21

2.24 Вариант 24 21

2.25 Вариант 25 22
2.26 Вариант 26 22
2.27 Вариант 27 23
2.28 Вариант 28 23
2.29 Вариант 29 24
2.30 Вариант 30 24

3 Приложения **24**
Приложение А. Генераторы синтетических данных 24
Приложение Б. Вспомогательный код для шага 3.4 26
Приложение В. Версии библиотек 28

1 Описание задания

Цель работы

Решить задачу классификации текста двумя способами — классическим методом (TF-IDF и логистическая регрессия) и дообучением (fine-tuning) предобученного трансформера DistilBERT, — сравнить их по качеству и времени обучения, по кривым обучения количественно оценить переобучение и провести один управляемый эксперимент, заданный вариантом.

Окружение

Python 3.10–3.12, библиотеки из приложения В. Работа выполняется в Jupyter-ноутбуке; желательна видеокарта (подойдёт бесплатный Google Colab с T4). На видеокарте с 4 ГБ памяти одно дообучение занимает 1–4 минуты (вариант 4 — до 7 минут), на процессоре — в 10–30 раз дольше.

Общие требования

- Номер варианта и зерно генератора случайных чисел задаются в первой ячейке ноутбука; зерно передаётся во все функции, где есть случайность.
- Данные варианта приводятся к набору `pool` с двумя колонками: `text` и `label`. Из него берётся не более 5000 примеров и делится на обучающую, валидационную и тестовую части в пропорции 70/15/15.
- Все сравнения выполняются на одной и той же тестовой части.
- Отчёт — исполненный ноутбук в формате «код → таблица или график → вывод»; вывод — 3–5 предложений с числами.

Первая ячейка ноутбука и функция приведения данных к виду `text`/`label`:

```
VARIANT = 1                     # номер своего варианта, 1..30
SEED = 1000 + VARIANT
EPOCHS, LR = 5, 2e-5             # варианты 13 и 14 переопределяют в своём коде

import random, time
import numpy as np
import torch
from datasets import load_dataset, concatenate_datasets

random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
```

```
def to_pool(parts, text_fn, label_fn):
    """Склеивает части датасета и приводит их к двум колонкам: text, label."""
    d = concatenate_datasets(parts)
    d = d.map(lambda ex: {"text": text_fn(ex), "label": label_fn(ex)},
              remove_columns=d.column_names)
    return d.filter(lambda ex: bool(ex["text"]) and ex["label"] != -1)
```

После кода своего варианта (раздел 2) данные делятся на части:

```
# pool, LABELS, MAX_LEN заданы кодом варианта раздел( 2)
N_MAX = 2000 if VARIANT == 28 else 5000
pool = pool.shuffle(seed=SEED).select(range(min(N_MAX, len(pool))))
a = pool.train_test_split(test_size=0.30, seed=SEED)
b = a["test"].train_test_split(test_size=0.50, seed=SEED)
train_ds, val_ds, test_ds = a["train"], b["train"], b["test"]
print(len(train_ds), len(val_ds), len(test_ds))
```

1.1 Задание 1. Разведка данных и классический baseline

1. Постройте распределение классов в обучающей части и гистограмму длин текстов в токенах токенизатора DistilBERT. Укажите, какая доля текстов длиннее `MAX_LEN` своего варианта (эти тексты будут обрезаны).
2. Обучите конвейер TF-IDF (`max_features=10000`) $\rightarrow$ логистическая регрессия (`max_iter=1000`). Измерьте время обучения в секундах, Accuracy и Macro-F1 на тестовой части.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score

t0 = time.time()
vec = TfidfVectorizer(max_features=10000)
X_train = vec.fit_transform(train_ds["text"])
X_test = vec.transform(test_ds["text"])
clf = LogisticRegression(max_iter=1000).fit(X_train, train_ds["label"])
pred = clf.predict(X_test)
print(f"время обучения: {time.time() - t0:.1f} c")
print("Accuracy:", accuracy_score(test_ds["label"], pred))
print("Macro-F1:", f1_score(test_ds["label"], pred, average="macro"))
```

Что сдать: распределение классов, гистограмма длин, доля обрезаемых текстов, метрики и время обучения baseline.

1.2 Задание 2. Дообучение предобученного трансформера

Дообучите модель `distilbert-base-uncased` с классификационной головой. Длина входа — `MAX_LEN` варианта, число эпох и скорость обучения — `EPOCHS` и `LR` (по умолчанию 5 эпох и $2 \cdot 10^{-5}$). Валидация проводится после каждой эпохи; лучшая по Macro-F1 модель сохраняется, обучение останавливается, если Macro-F1 на валидации не растёт две эпохи подряд.

```
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer, DataCollatorWithPadding,
                          EarlyStoppingCallback)

model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=len(LABELS))


def encode(d):
    d = d.map(lambda b: tokenizer(b["text"], truncation=True, max_length=MAX_LEN),
              batched=True)
    return d.rename_column("label", "labels").remove_columns(["text"])


train_enc, val_enc, test_enc = encode(train_ds), encode(val_ds), encode(test_ds)


def compute_metrics(p):
    pred = p.predictions.argmax(-1)
    return {"accuracy": accuracy_score(p.label_ids, pred),
            "f1_macro": f1_score(p.label_ids, pred, average="macro")}


args = TrainingArguments(
    output_dir="./out", num_train_epochs=EPOCHS, learning_rate=LR,
    per_device_train_batch_size=16, per_device_eval_batch_size=64,
    eval_strategy="epoch", save_strategy="epoch", logging_strategy="epoch",
    load_best_model_at_end=True, metric_for_best_model="f1_macro",
    save_total_limit=1, seed=SEED, report_to="none",
    fp16=torch.cuda.is_available())
trainer = Trainer(
```

```
    model=model, args=args, train_dataset=train_enc, eval_dataset=val_enc,
    processing_class=tokenizer, data_collator=DataCollatorWithPadding(tokenizer),
    compute_metrics=compute_metrics,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=2)])
t0 = time.time()
trainer.train()
print(f"время обучения: {time.time() - t0:.1f} с")
print(trainer.evaluate(test_enc, metric_key_prefix="test"))

# кривые обучения: train loss и val loss по эпохам
hist = trainer.state.log_history
train_loss = [h["loss"] for h in hist if "loss" in h]
val_loss = [h["eval_loss"] for h in hist if "eval_loss" in h]
```

Измерьте Accuracy, Macro-F1 и время обучения лучшей модели на тестовой части. Постройте график train loss и val loss по эпохам.

Материал: лекции 1–2 (токенизация, энкодер, предобученные модели); лекция 4, раздел «Средства против переобучения» (ранняя остановка).

Что сдать: график кривых обучения; таблица «модель × Accuracy × Macro-F1 × время» для baseline и DistilBERT.

1.3 Задание 3. Анализ переобучения и ошибок

Типичная динамика потерь при переобучении (пример):

Эпоха	Train loss	Val loss
1	0.95	0.62
2	0.55	0.48 (минимум)
3	0.32	0.51
4	0.18	0.66
5	0.12	0.85

Шаг 3.1. По своим кривым из задания 2 определите: на какой эпохе минимум val loss и на какой — максимум val Macro-F1 (они могут не совпадать); каков разрыв val loss – train loss на последней эпохе (количественная мера переобучения); сработала ли ранняя остановка и на какой эпохе. Сравните свою картину с таблицей выше.

Материал: лекция 4, слайд «Переобучение: динамика по эпохам».

Шаг 3.2. Постройте матрицу ошибок (`ConfusionMatrixDisplay` из `sklearn.metrics`) на тестовой части. Назовите пару классов, которые модель путает чаще всего, и объясните почему. Для вариантов со 100 и более классами (13, 14) приведите 10 самых частых пар вместо матрицы; для варианта 22 — F1 по каждой метке.

Шаг 3.3. Разберите 3 ошибки модели: текст, истинная метка, предсказанные вероятности классов. Сформулируйте гипотезу причины каждой ошибки.

Шаг 3.4. Проведите эксперимент своего варианта (раздел 2). Во всех вариантах, кроме 22, эксперимент требует ровно одного дополнительного обучения с теми же настройками, что в задании 2, если в варианте не сказано иное. Результат оформите таблицей «постановка × Accuracy × Macro-F1 × время» и объясните разницу. Вспомогательный код (веса классов, multi-label, регрессия) — в приложении Б.

Что сдать: ответы на вопросы шага 3.1 с числами, матрица ошибок и разбор трёх ошибок, таблица и вывод эксперимента шага 3.4.

1.4 Содержание отчёта и оценивание

Отчёт — исполненный ноутбук: все ячейки выполнены, графики подписаны (оси, легенда), после каждого задания — вывод с числами.

Критерий	Вес
Задание 1: статистика данных, baseline с метриками и временем	25%
Задание 2: модель дообучена, кривые обучения, сравнение с baseline	30%
Задание 3: анализ переобучения и ошибок, эксперимент и его интерпретация	30%
Оформление и качество выводов	15%

2 Данные для вариантов

Таблица 1: Варианты заданий (K — число классов, max_len — длина входа в токенах)

№	Датасет	Задача	K	max_len	Эксперимент (шаг 3.4)
1	nyu-mll/glue, sst2	Тональность	2	64	max_len 64 и 16
2	cornell-movie-review-data/rotten_tomatoes	Тональность	2	64	max_len 64 и 32
3	stanfordnlp/imdb	Тональность	2	128	Начало или конец отзыва
4	stanfordnlp/imdb	Тональность	2	256	max_len 256 и 128, время
5	cardiffnlp/tweet_eval, sentiment	Тональность твитов	3	32	Удаление @user и ссылок
6	Yelp/yelp_review_full	Оценка отзыва, 1–5 звёзд	5	128	Соседние классы; 3 класса
7	fancyzhx/amazon_polarity	Тональность	2	128	Баланс 50/50 и 90/10
8	fancyzhx/ag_news	Тема новости	4	64	Путаница Business/Sci/Tech
9	fancyzhx/dbpedia_14	Тип статьи	14	64	14 классов и 4 класса
10	community-datasets/yahoo_answers_topics	Тема вопроса	10	64	Только заголовок вопроса
11	dair-ai/emotion	Эмоция	6	32	Веса классов
12	google-research-datasets/go_emotions, simplified	Эмоция joy / нет	2	64	Выбор целевой эмоции
13	clinc/clinc_oos, plus	Интент или «вне темы» (oos)	151	32	OOS: класс или порог
14	legacy-datasets/banking77	Интент банковского бота	77	32	Недообучение; macro/weighted F1
15	financial_phrasebank, allagree	Тональность новостей	3	64	Без нейтрального класса
16	financial_phrasebank, allagree	Тональность новостей	3	64	Разметка allagree и 50agree
17	TimSchopf/medical_abstracts	Раздел медицины по аннотации	5	128	Только заголовок статьи

№	Датасет	Задача	K	max_len	Эксперимент (шаг 3.4)
18	SetFit/ethos_binary	Язык вражды	2	64	Ложные срабатывания; веса
19	SetFit/sst5	Тональность, 5 уровней	5	64	Классификация и регрессия
20	CogComp/trec	Тип вопроса	6	32	Только первое слово
21	cardiffnlp/tweet_eval, irony	Ирония	2	32	Ошибки; max_len 64
22	SemEval-2018 Task 1 (E-c)	11 эмоций, несколько меток	11	64	Подбор порогов
23	toxigen/toxigen-data, annotated	Токсичность	2	64	Цена ложного срабатывания
24	mteb/amazon_massive_scenario, en	Сценарий ассистента	18	32	20 примеров на класс
25	ucirvine/sms_spam	Спам	2	64	Дисбаланс 1:6.5 и 1:20
26	синтетика (приложение А)	Тональность с шумом в метках	2	64	Шум 20% и 5%
27	fancyzhx/ag_news	Спорт / остальное	2	64	Потолок качества: 10% данных
28	stanfordnlp/imdb, срез 2000	Тональность	2	128	2000 и 5000 примеров
29	dair-ai/emotion	Эмоция	6	32	Больше данных или эпох
30	синтетика (приложение А)	Интент банковского бота	3	32	Обобщение на новые шаблоны

Код варианта вставляется в ноутбук после первой ячейки: он загружает датасет, собирает `pool` и задаёт `LABELS` (названия классов по порядку номеров) и `MAX_LEN`. Датасеты загружаются с Hugging Face Hub по полному имени `автор/название`: короткие имена (`"imdb"`, `"glue"`) и датасеты со скриптами загрузки текущие версии библиотеки `datasets` не поддерживают, поэтому

для части датасетов указаны копии в формате parquet.

2.1 Вариант 1

Дано: датасет nyu-mll/glue, sst2; задача — тональность; классов: 2; `MAX_LEN` = 64.

```
ds = load_dataset("nyu-mll/glue", "sst2")          # y test метки скрыты (-1)
pool = to_pool([ds["train"], ds["validation"]], lambda ex: ex["sentence"], lambda ex:
    ex["label"])
LABELS = ds["train"].features["label"].names         # ['negative', 'positive']
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Длина входа: обучите модель с `MAX_LEN = 16` и сравните с основной (64). Какая доля текстов обрезается при 16 токенах?

2.2 Вариант 2

Дано: датасет cornell-movie-review-data/rotten_tomatoes; задача — тональность; классов: 2; `MAX_LEN` = 64.

```
ds = load_dataset("cornell-movie-review-data/rotten_tomatoes")
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ds["train"].features["label"].names         # ['neg', 'pos']
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Длина входа: обучите модель с `MAX_LEN = 32` и сравните с основной (64). Какая доля текстов обрезается при 32 токенах?

2.3 Вариант 3

Дано: датасет stanfordnlp/imdb; задача — тональность; классов: 2; `MAX_LEN` = 128.

```
ds = load_dataset("stanfordnlp/imdb")                # часть unsupervised без меток не
    берём
pool = to_pool([ds["train"], ds["test"]], lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ds["train"].features["label"].names         # ['neg', 'pos']
MAX_LEN = 128
```

Эксперимент (шаг 3.4). Начало или конец отзыва: обучите вторую модель на последних 128 токенах отзыва (`tokenizer.truncation_side = "left"` перед токенизацией) и сравните с основной, которая видит первые 128 токенов.

2.4 Вариант 4

Дано: датасет stanfordnlp/imdb; задача — тональность; классов: 2; `MAX_LEN` = 256.

```
ds = load_dataset("stanfordnlp/imdb")                # часть unsupervised без меток не
    берём
pool = to_pool([ds["train"], ds["test"]], lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ds["train"].features["label"].names         # ['neg', 'pos']
MAX_LEN = 256
```

Эксперимент (шаг 3.4). Длина входа и время: обучите модель с `MAX_LEN = 128` и сравните с основной (256) по качеству и времени обучения.

2.5 Вариант 5

Дано: датасет cardiffnlp/tweet_eval, sentiment; задача — тональность твитов; классов: 3; `MAX_LEN` = 32.

```
ds = load_dataset("cardiffnlp/tweet_eval", "sentiment")
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ds["train"].features["label"].names         # ['negative', 'neutral',
    'positive']
MAX_LEN = 32
```

Эксперимент (шаг 3.4). Предобработка: удалите из всех текстов упоминания `@user` и ссылки (`re.sub(r"(@user|http\S*)", text)`), переобучите модель и сравните. Какая доля твитов содержала упоминания или ссылки?

2.6 Вариант 6

Дано: датасет Yelp/yelp_review_full; задача — оценка отзыва, 1–5 звёзд; классов: 5; `MAX_LEN` = 128.

```
ds = load_dataset("Yelp/yelp_review_full")
pool = to_pool([ds["train"]], lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ds["train"].features["label"].names         # ['1 star', ..., '5 stars']
MAX_LEN = 128
```

Эксперимент (шаг 3.4). Путаница соседних классов: посчитайте, какая доля ошибок приходится на соседний класс (ошибка на одну звезду). Обучите модель на трёх классах (1–2, 3 и 4–5 звёзд) и сравните.

2.7 Вариант 7

Дано: датасет fancyzhx/amazon_polarity; задача — тональность; классов: 2; `MAX_LEN` = 128.

```
ds = load_dataset("fancyzhx/amazon_polarity")
pool = to_pool([ds["train"]], lambda ex: ex["title"] + ". " + ex["content"], lambda ex:
    ex["label"])
LABELS = ds["train"].features["label"].names         # ['negative', 'positive']
MAX_LEN = 128
```

Эксперимент (шаг 3.4). Дисбаланс: оставьте в обучающей части все отрицательные отзывы и столько положительных, чтобы их соотношение было 90/10; валидацию и тест не меняйте. Сравните Accuracy, Macro-F1 и полноту положительного класса.

2.8 Вариант 8

Дано: датасет fancyzhx/ag_news; задача — тема новости; классов: 4; `MAX_LEN` = 64.

```
ds = load_dataset("fancyzhx/ag_news")
pool = to_pool([ds["train"]], lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ds["train"].features["label"].names         # ['World', 'Sports', 'Business',
    'Sci/Tech']
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Путаница Business и Sci/Tech: выпишите число ошибок в обе стороны и 3 примера. Обучите модель на первых 25% обучающей части и проверьте, как изменились качество и эта путаница.

2.9 Вариант 9

Дано: датасет fancyzhx/dbpedia_14; задача — тип статьи; классов: 14; `MAX_LEN` = 64.

```
ds = load_dataset("fancyzhx/dbpedia_14")
pool = to_pool([ds["train"]], lambda ex: ex["title"] + ". " + ex["content"], lambda ex:
    ex["label"])
LABELS = ds["train"].features["label"].names         # 14 классов
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Число классов и скорость обучения: оставьте в выборках только классы 0–3, обучите модель и сравните с основной по val Macro-F1 после каждой эпохи, числу эпох до ранней остановки и времени.

2.10 Вариант 10

Дано: датасет community-datasets/yahoo_answers_topics; задача — тема вопроса; классов: 10; `MAX_LEN` = 64.

```
ds = load_dataset("community-datasets/yahoo_answers_topics")
full_text = lambda ex: " ".join(x for x in (ex["question_title"],
    ex["question_content"], ex["best_answer"]) if x)
pool = to_pool([ds["train"]], full_text, lambda ex: ex["topic"])
LABELS = ds["train"].features["topic"].names         # 10 тем
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Состав текста: соберите `pool` заново только из заголовка вопроса (`question_title`), разделите его так же и обучите модель. Насколько полезны текст вопроса и лучший ответ?

2.11 Вариант 11

Дано: датасет dair-ai/emotion; задача — эмоция; классов: 6; `MAX_LEN` = 32.

```
ds = load_dataset("dair-ai/emotion")
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ds["train"].features["label"].names         # ['sadness', 'joy', 'love',
    'anger', 'fear', 'surprise']
MAX_LEN = 32
```

Эксперимент (шаг 3.4). Веса классов: классы несбалансированы. Обучите модель со взвешенной функцией потерь (приложение Б) и сравните Macro-F1 и F1 редких классов.

2.12 Вариант 12

Дано: датасет google-research-datasets/go_emotions, simplified; задача — эмоция joy / нет; классов: 2; `MAX_LEN` = 64.

```
ds = load_dataset("google-research-datasets/go_emotions", "simplified")
EMO = ds["train"].features["labels"].feature.names.index("joy")   # шаг 3.4: "gratitude"
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex: int(EMO in
    ex["labels"]))
LABELS = ["other", "joy"]
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Выбор целевой эмоции: замените `"joy"` на `"gratitude"`, соберите данные заново и обучите модель. Сравните долю положительного класса и Macro-F1 для двух эмоций и объясните разницу.

2.13 Вариант 13

Дано: датасет clinc/clinc_oos, plus; задача — интент или «вне темы» (oos); классов: 151; `MAX_LEN` = 32.

```
ds = load_dataset("clinc/clinc_oos", "plus")
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex: ex["intent"])
LABELS = ds["train"].features["intent"].names        # 150 интентов + 'oos'
MAX_LEN = 32
EPOCHS, LR = 10, 5e-5                                 # 151 класс: при 5 эпохах и 2e-5
    модель недообучается
```

Эксперимент (шаг 3.4). Запросы вне темы (OOS): в основной модели `oos` — отдельный класс. Обучите модель только на 150 интентах (без `oos`) и считайте запрос OOS, если максимальная вероятность класса ниже порога 0.3, 0.5 или 0.7. Сравните с основной моделью полноту обнаружения OOS и долю обычных запросов, ошибочно отнесённых к OOS.

2.14 Вариант 14

Дано: датасет legacy-datasets/banking77; задача — интент банковского бота; классов: 77; `MAX_LEN` = 32.

```
ds = load_dataset("legacy-datasets/banking77")
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ds["train"].features["label"].names         # 77 интентов
MAX_LEN = 32
EPOCHS, LR = 10, 5e-5                                 # шаг 3.4 сравнивает с 5 эпохами и
    2e-5
```

Эксперимент (шаг 3.4). Недообучение и выбор метрики: обучите модель с настройками по умолчанию (5 эпох, $2 \cdot 10^{-5}$) и сравните с основной (10 эпох, $5 \cdot 10^{-5}$). Для основной модели посчитайте macro- и weighted-F1 и объясните, когда они расходятся.

2.15 Вариант 15

Дано: датасет financial_phrasebank, allagree; задача — тональность новостей; классов: 3; `MAX_LEN` = 64.

```
# зеркало financial_phrasebank (sentences_allagree): оригинал загружается только скриптом
ds = load_dataset("gtfintechlab/financial_phrasebank_sentences_allagree", "5768")
pool = to_pool(list(ds.values()), lambda ex: ex["sentence"], lambda ex: ex["label"])
LABELS = ["negative", "neutral", "positive"]
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Бинаризация: удалите нейтральный класс из всех частей, перенумеруйте метки (negative — 0, positive — 1) и обучите модель на двух классах. Сравните и объясните, почему нейтральный класс труднее.

2.16 Вариант 16

Дано: датасет financial_phrasebank, allagree; задача — тональность новостей; классов: 3; `MAX_LEN` = 64.

```
# зеркало financial_phrasebank (sentences_allagree): оригинал загружается только скриптом
ds = load_dataset("gtfintechlab/financial_phrasebank_sentences_allagree", "5768")
pool = to_pool(list(ds.values()), lambda ex: ex["sentence"], lambda ex: ex["label"])
LABELS = ["negative", "neutral", "positive"]
MAX_LEN = 64
# шаг 3.4: разметка с согласием >= 50% аннотаторов те( же метки 0/1/2)
ds50 = load_dataset("warwickai/financial_phrasebank_mirror")
```

Эксперимент (шаг 3.4). Качество разметки: в основной модели все аннотаторы согласны с меткой. Возьмите из `ds50` (согласие не менее 50% аннотаторов) столько же примеров, сколько в обучающей части, исключив предложения своих валидации и теста, обучите модель и оцените на том же тесте.

2.17 Вариант 17

Дано: датасет TimSchopf/medical_abstracts; задача — раздел медицины по аннотации; классов: 5; `MAX_LEN` = 128.

```
ds = load_dataset("TimSchopf/medical_abstracts")    # метки 1..5 -> 0..4
pool = to_pool(list(ds.values()), lambda ex: ex["medical_abstract"], lambda ex:
    ex["condition_label"] - 1)
LABELS = ["neoplasms", "digestive", "nervous", "cardiovascular", "general"]
MAX_LEN = 128
```

Эксперимент (шаг 3.4). Специфичная лексика и заголовок: аннотация начинается с заголовка статьи. Обучите модель только на заголовке (`text.split(". ")[0]`) и сравните с основной. Какая доля слов аннотаций разбивается токенизатором на 3 и более подтокена?

2.18 Вариант 18

Дано: датасет SetFit/ethos_binary; задача — язык вражды; классов: 2; `MAX_LEN` = 64.

```
ds = load_dataset("SetFit/ethos_binary")
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ["not_hate", "hate"]
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Ложные срабатывания: посчитайте FP и FN основной модели и разберите 3 ложноположительных примера. Обучите модель с весами классов (приложение Б) и сравните FP и FN. Всего в датасете около 1000 примеров.

2.19 Вариант 19

Дано: датасет SetFit/sst5; задача — тональность, 5 уровней; классов: 5; `MAX_LEN` = 64.

```
ds = load_dataset("SetFit/sst5")
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ["very negative", "negative", "neutral", "positive", "very positive"]
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Классификация или регрессия: обучите модель, предсказывающую одно число (приложение Б), округлите предсказание до ближайшего класса и сравните Accuracy и Macro-F1 с классификацией.

2.20 Вариант 20

Дано: датасет CogComp/trec; задача — тип вопроса; классов: 6; `MAX_LEN` = 32.

```
# оригинал загружается только скриптом; берём его копию в формате parquet
ds = load_dataset("CogComp/trec", revision="refs/convert/parquet")
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex: ex["coarse_label"])
LABELS = ds["train"].features["coarse_label"].names  # ['ABBR', 'ENTY', 'DESC', 'HUM',
    'LOC', 'NUM']
MAX_LEN = 32
```

Эксперимент (шаг 3.4). Первое слово: оставьте в каждом тексте только первое слово (`text.split()[0]`) и обучите модель. Какую долю качества даёт одно вопросительное слово?

2.21 Вариант 21

Дано: датасет cardiffnlp/tweet_eval, irony; задача — ирония; классов: 2; `MAX_LEN` = 32.

```
ds = load_dataset("cardiffnlp/tweet_eval", "irony")
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ds["train"].features["label"].names         # ['non_irony', 'irony']
MAX_LEN = 32
```

Эксперимент (шаг 3.4). Разбор ошибок иронии: посчитайте FP и FN, разберите 5 ошибок и сгруппируйте их по причинам. Обучите модель с `MAX_LEN = 64` и сравните.

2.22 Вариант 22

Дано: датасет SemEval-2018 Task 1 (E-c); задача — 11 эмоций, несколько меток; классов: 11; `MAX_LEN` = 64.

```
# SemEval-2018 Task 1 (E-c), английская часть; оригинал загружается только скриптом
ds = load_dataset("vibhorag101/sem_eval_2018_task_1_english_cleaned_labels")
LABELS = ["anger", "anticipation", "disgust", "fear", "joy", "love",
          "optimism", "pessimism", "sadness", "surprise", "trust"]
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex: [float(ex[k]) for k
    in LABELS])
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Порог решения: сравните macro-F1 при пороге 0.5 и при порогах, подобранных по валидации для каждой метки отдельно (сетка 0.05–0.95). Дополнительное обучение не требуется: в этом варианте сложнее основная часть — модель с несколькими метками (приложение Б).

2.23 Вариант 23

Дано: датасет toxigen/toxigen-data, annotated; задача — токсичность; классов: 2; `MAX_LEN` = 64.

```
ds = load_dataset("toxigen/toxigen-data", "annotated")
# toxicity_human -- средняя оценка аннотаторов по шкале 1..5; токсичным считаем >= 3
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex:
    int(ex["toxicity_human"] >= 3))
LABELS = ["benign", "toxic"]
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Цена ложного срабатывания: считая ложное срабатывание (FP) в 5 раз дороже пропуска (FN), подберите по вероятности класса `toxic` порог с минимальной стоимостью. Обучите модель с весами классов (приложение Б) и сравните FP, FN и стоимость.

2.24 Вариант 24

Дано: датасет mteb/amazon_massive_scenario, en; задача — сценарий ассистента; классов: 18; `MAX_LEN` = 32.

```
ds = load_dataset("mteb/amazon_massive_scenario", "en")
LABELS = sorted(set(ds["train"]["label"]))          # 18 сценариев голосового ассистента
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex:
    LABELS.index(ex["label"]))
MAX_LEN = 32
```

Эксперимент (шаг 3.4). Мало данных: оставьте в обучающей части по 20 случайных примеров каждого класса, обучите модель и сравните с основной и с TF-IDF на тех же данных.

2.25 Вариант 25

Дано: датасет ucirvine/sms_spam; задача — спам; классов: 2; `MAX_LEN` = 64.

```
ds = load_dataset("ucirvine/sms_spam")
pool = to_pool([ds["train"]], lambda ex: ex["sms"], lambda ex: ex["label"])
LABELS = ["ham", "spam"]
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Дисбаланс: в данных спама примерно в 6.5 раз меньше, чем обычных сообщений. Оставьте в обучающей части столько спама, чтобы соотношение было 1:20, обучите модель и сравните F1 и полноту класса `spam` на том же тесте.

2.26 Вариант 26

Дано: датасет синтетика (приложение А); задача — тональность с шумом в метках; классов: 2; `MAX_LEN` = 64.

```
# make_noisy_sentiment_dataset -- приложение А
pool = make_noisy_sentiment_dataset(flip_prob=0.2)   # шаг 3.4: flip_prob=0.05
LABELS = ["negative", "positive"]
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Уровень шума: сгенерируйте данные с `flip_prob=0.05`, обучите модель и сравните с шумом 20%. Какое максимальное качество возможно при каждом уровне шума и почему TF-IDF не уступает трансформеру?

2.27 Вариант 27

Дано: датасет fancyzhx/ag_news; задача — спорт / остальное; классов: 2; `MAX_LEN` = 64.

```
ds = load_dataset("fancyzhx/ag_news")
pool = to_pool([ds["train"]], lambda ex: ex["text"], lambda ex: int(ex["label"] == 1))
LABELS = ["other", "sports"]
MAX_LEN = 64
```

Эксперимент (шаг 3.4). Потолок качества: задача простая. Обучите модель на первых 10% обучающей части и сравните. Сколько данных нужно, чтобы выйти на потолок? Разберите оставшиеся ошибки основной модели.

2.28 Вариант 28

Дано: датасет stanfordnlp/imdb, срез 2000; задача — тональность; классов: 2; `MAX_LEN` = 128.

```
ds = load_dataset("stanfordnlp/imdb")               # часть unsupervised без меток не
    берём
pool = to_pool([ds["train"], ds["test"]], lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ds["train"].features["label"].names        # ['neg', 'pos']
MAX_LEN = 128
```

Эксперимент (шаг 3.4). Объём данных: соберите разбиение из 5000 примеров, предварительно исключив из `pool` тексты своего теста, обучите модель и оцените на том же тесте. Сравните кривые обучения: где переобучение сильнее?

2.29 Вариант 29

Дано: датасет dair-ai/emotion; задача — эмоция; классов: 6; `MAX_LEN` = 32.

```
ds = load_dataset("dair-ai/emotion")
pool = to_pool(list(ds.values()), lambda ex: ex["text"], lambda ex: ex["label"])
LABELS = ds["train"].features["label"].names         # ['sadness', 'joy', 'love',
    'anger', 'fear', 'surprise']
MAX_LEN = 32
```

Эксперимент (шаг 3.4). Данные или эпохи: обучите модель на первой половине обучающей части в течение 10 эпох без ранней остановки (`callbacks=[]`) и сравните с основной (все данные, до 5 эпох). Что полезнее при равном числе шагов?

2.30 Вариант 30

Дано: датасет синтетика (приложение А); задача — интент банковского бота; классов: 3; `MAX_LEN` = 32.

```
# make_template_intent_dataset -- приложение А
full = make_template_intent_dataset()
pool = full.remove_columns(["template_id", "split"])  # основная постановка: случайный
    сплит
LABELS = ["balance", "transfer", "block_card"]
MAX_LEN = 32
```

Эксперимент (шаг 3.4). Обобщение на новые формулировки: в основной постановке шаблоны тестовых фраз встречаются в обучении. Обучите модель только на примерах `full` с `split == "seen"` (часть из них — валидация) и проверьте на примерах с `split == "unseen"` (шаблоны, которых модель не видела). Какая доля тестовых текстов основной постановки дословно встречается в обучающей части?

3 Приложения

Приложение А. Генераторы синтетических данных (варианты 26, 30)

```
import random
from datasets import Dataset
```

```
def make_noisy_sentiment_dataset(n=3000, seed=1026, flip_prob=0.2):
    """ЛР1, вариант 26: тональность с шумом в разметке доля( перевёрнутых меток
    flip_prob)."""
    rng = random.Random(seed)
    pos = ["great product", "love it", "excellent quality", "very satisfied", "works
    perfectly"]
    neg = ["terrible experience", "waste of money", "broken on arrival", "awful
    service", "do not buy"]
    texts, labels = [], []
    for _ in range(n):
        y = rng.randint(0, 1)
        s = rng.choice(pos if y == 1 else neg)
        if rng.random() < flip_prob:
            y = 1 - y
        texts.append(s); labels.append(y)
    return Dataset.from_dict({"text": texts, "label": labels})


BANK_TEMPLATES = {
    0: ["what is my {acc} balance", "how much money is left on my {acc}",
        "show the remaining funds on my {acc}", "can you tell me what i have on my
    {acc}",
        "check my {acc} please", "i want to know my {acc} total",
        "how much do i have in my {acc} right now", "tell me the amount available on my
    {acc}"],
    1: ["send {amt} dollars to {name}", "transfer {amt} to {name} from my {acc}",
        "i need to pay {name} {amt} dollars", "move {amt} from my {acc} to {name}",
        "wire {amt} to {name}", "please give {name} {amt} from my {acc}",
        "can i send {name} {amt} today", "make a payment of {amt} to {name}"],
    2: ["block my {card}", "my {card} was stolen", "i lost my {card} yesterday",
        "freeze the {card} right away", "someone took my {card}", "disable my {card}
    please",
        "i can not find my {card} anywhere", "stop all payments on my {card}"],
}


def make_template_intent_dataset(n=3000, seed=1030, n_train_templates=5):
    """ЛР1, вариант 30: интенты банковского бота из шаблонов.
    Поле template_id позволяет разбить данные так, чтобы шаблоны теста не встречались в
    обучении:
    шаблоны с номерами < n_train_templates — 'seen', остальные — 'unseen'."""
    rng = random.Random(seed)
    accs = ["checking account", "savings account", "account", "deposit", "wallet"]
    cards = ["card", "credit card", "debit card", "visa card", "bank card"]
```

```
    names = ["anna", "john", "maria", "peter", "olga", "david", "lena", "mike"]
    texts, labels, tids = [], [], []
    for _ in range(n):
        y = rng.randint(0, 2)
        t = rng.randrange(len(BANK_TEMPLATES[y]))
        s = (BANK_TEMPLATES[y][t].replace("{acc}", rng.choice(accs)).replace("{card}",
    rng.choice(cards))
             .replace("{name}", rng.choice(names)).replace("{amt}", str(rng.randint(5,
    500))))
        texts.append(s); labels.append(y); tids.append(t)
    split = ["seen" if t < n_train_templates else "unseen" for t in tids]
    return Dataset.from_dict({"text": texts, "label": labels, "template_id": tids,
    "split": split})
```

Приложение Б. Вспомогательный код для шага 3.4

Веса классов (варианты 11, 18, 23): взвешенная кросс-энтропия, вес класса k равен $N/(K \cdot N_k)$, где N — размер обучающей части, K — число классов, N_k — число примеров класса k. Код выполняется после кода задания 2.

```
from collections import Counter


class WeightedTrainer(Trainer):
    """Trainer с взвешенной функцией потерь cross_entropy: вес класса k = N / (K *
    N_k)."""
    def __init__(self, *a, class_weights=None, **kw):
        super().__init__(*a, **kw)
        self.class_weights = class_weights

    def compute_loss(self, model, inputs, return_outputs=False, **kw):
        labels = inputs.pop("labels")
        out = model(**inputs)
        loss = torch.nn.functional.cross_entropy(
            out.logits, labels, weight=self.class_weights.to(out.logits.device))
        return (loss, out) if return_outputs else loss


cnt = Counter(train_ds["label"])
K = len(LABELS)
w = torch.tensor([len(train_ds) / (K * cnt[k]) for k in range(K)], dtype=torch.float32)
model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=K)
trainer = WeightedTrainer(
```

```
    model=model, args=args, train_dataset=train_enc, eval_dataset=val_enc,
    processing_class=tokenizer, data_collator=DataCollatorWithPadding(tokenizer),
    compute_metrics=compute_metrics, class_weights=w,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=2)])
trainer.train()
print(trainer.evaluate(test_enc, metric_key_prefix="test"))
```

Несколько меток на пример (вариант 22): модель и метрики вместо соответствующих строк задания 2; порог 0.5 применяется к вероятности каждой метки отдельно.

```
# Вариант 22: несколько меток на пример (multi-hot вектор из 0.0 / 1.0)
model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=len(LABELS), problem_type="multi_label_classification")


def compute_metrics(p):
    prob = 1 / (1 + np.exp(-p.predictions))
    pred, y = (prob >= 0.5).astype(int), p.label_ids.astype(int)
    return {"accuracy": accuracy_score(y, pred),      # доля полностью угаданных наборов
            "f1_macro": f1_score(y, pred, average="macro", zero_division=0)}

# baseline для задания 1: один классификатор на метку
# from sklearn.multiclass import OneVsRestClassifier
# clf = OneVsRestClassifier(LogisticRegression(max_iter=1000))
# clf.fit(X_train, np.array(train_ds["label"]).astype(int))
```

Регрессия вместо классификации (вариант 19, шаг 3.4): выполняется после функции `encode` задания 2, затем создаётся `Trainer` с этими `model` и `compute_metrics`. Для выбора лучшей модели в `TrainingArguments` оставьте `metric_for_best_model="f1_macro"`.

```
# Вариант 19, шаг 3.4: та же задача как регрессия одно( число на выходе)
model = AutoModelForSequenceClassification.from_pretrained(
    model_name, num_labels=1, problem_type="regression")
from datasets import Value
to_float = lambda d: d.cast_column("labels", Value("float32"))
train_enc, val_enc, test_enc = to_float(train_enc), to_float(val_enc), to_float(test_enc)


def compute_metrics(p):
    pred = np.clip(np.rint(p.predictions.squeeze(-1)), 0, len(LABELS) - 1).astype(int)
    y = np.rint(p.label_ids).astype(int)
    return {"accuracy": accuracy_score(y, pred),
            "f1_macro": f1_score(y, pred, average="macro"),
```

```
            "mae": float(np.abs(p.predictions.squeeze(-1) - p.label_ids).mean())}
```

Приложение В. Версии библиотек

Код задания проверен со следующими версиями (`requirements.txt`); с другими версиями возможны расхождения в именах аргументов:

```
torch==2.11.0
transformers==5.17.0
datasets==5.0.1
accelerate==1.15.0
scikit-learn==1.9.1
numpy==2.5.3
matplotlib==3.11.2
```